

Intrinsically Typed Syntax That Works (Functional Pearl)

ANONYMOUS AUTHOR(S)

Intrinsically typed syntax promises elegant mechanizations: well-typed terms are represented by well-typed data, substitution preserves typing by construction, and type safety proofs collapse to straightforward induction. For simple calculi such as the simply-typed lambda calculus this promise is fulfilled. For realistic systems, however, the technique often becomes impractical. In dependently typed proof assistants, intrinsically typed encodings of polymorphic calculi lead to pervasive type-changing equalities: renamings and substitutions indexed by other substitutions force proofs to transport terms across equal types that are not definitionally equal. This phenomenon—informally known as transfer hell—causes even routine metatheoretic arguments to be dominated by administrative reasoning.

This functional pearl shows how to make intrinsically typed syntax work in practice for polymorphic calculi. We give extracts of a mechanization of the metatheory of System F and extend it to System $F\omega$ in Agda while avoiding transfer reasoning almost entirely. The key idea is to internalize the equational theory of the σ -calculus for substitutions as definitional equality. We rely on proved σ -calculus laws for a functional representation of substitutions and install them into Agda's rewrite mechanism, obtaining canonical behavior for substitution and renaming without explicit transports. The main technical challenge is ensuring that the resulting rewrite system remains terminating and confluent together with Agda's built-in definitional equality.

The resulting development dramatically simplifies proofs: typing preservation becomes definitional, substitution lemmas disappear, and type conversion in System $F\omega$ can be handled intrinsically. Our experience suggests a general methodology: when intrinsically typed syntax generates transports, the correct solution is often not more lemmas but better definitional equality. We distill practical guidelines for mechanizing typed calculi in dependently typed proof assistants.

ACM Reference Format:

Anonymous Author(s). 2018. Intrinsically Typed Syntax That Works (Functional Pearl). *ACM Trans. Program. Lang. Syst.* 37, 4, Article 111 (August 2018), 16 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Intrinsically typed syntax is widely recommended for mechanizing type systems in dependently typed proof assistants. By representing well-typed terms as well-typed data, typing invariants hold by construction as in a Church-style presentation, i.e., making typing preservation trivial. For small calculi this approach works beautifully. However, for realistic polymorphic calculi such as System F, mechanizations often become complicated: conceptually simple proofs become cluttered with transports across equal types and administrative equalities.

The simply-typed lambda calculus is the prime example where intrinsically typed syntax works like a charm. Figure 1 shows typical definitions of the syntax of types, type contexts, variables, and expressions, which are indexed by contexts and types (left column). Building on these definitions it is straightforward to define a small-step operational semantics and proving type safety. This is because subject reduction holds by construction and progress follows by a simple inductive proof (right column). The style of these definitions is adapted from the textbook by [?]. We elide the definition of the substitution operator $_{-}$, as we discuss substitutions in detail in Section 2.1.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1558-4593/2018/8-ART111

<https://doi.org/XXXXXXX.XXXXXXX>

```

50 data Type : Set where
51   1 : Type
52   _⇒_ : Type → Type → Type
53
54 data Ctx : Set where
55   ∅ : Ctx
56   _▷_ : Ctx → Type → Ctx
57
58 data _⊃_ : Ctx → Type → Set where
59   here : (Γ ▷ T) ⊃ T
60   there : Γ ⊃ T → (Γ ▷ U) ⊃ T
61
62 data _⊢_ Γ : Type → Set where
63   con : Γ ⊢ 1
64   var : Γ ⊃ T → Γ ⊢ T
65   lam : (Γ ▷ T) ⊢ U → Γ ⊢ (T ⇒ U)
66   app : Γ ⊢ (T ⇒ U) → Γ ⊢ T → Γ ⊢ U
67
68
69
70 data _→_ {Γ} {T} : Γ ⊢ T → Γ ⊢ T → Set where
71   β-lam : app {Γ = Γ} (lam e1) e2 → (e1 [ e2 ])
72   ξ-app : e1 → e1' → app e1 e2 → app e1' e2
73
74 data Value {Γ} : Γ ⊢ T → Set where
75   con : Value con
76   lam : (e : (Γ ▷ T) ⊢ U) → Value (lam e)
77
78 data Progress : Γ ⊢ T → Set where
79   done : Value e → Progress e
80   step : e → e' → Progress e
81
82 progress : (e : ∅ ⊢ T) → Progress e
83 progress con = done con
84 progress (lam e) = done (lam e)
85 progress (app e1 e2)
86   with progress e1
87   ... | step x = step (ξ-app x)
88   ... | done (lam e) = step β-lam

```

Fig. 1. Call-by-name simply-typed lambda calculus

The simply-typed lambda calculus therefore represents the ideal scenario: intrinsic typing elides type preservation proofs, thus making type safety proofs so concise that they can serve as textbook examples [?]. Nothing comparable to transport reasoning arises in this development as only definitional properties of substitutions are required. The remainder of this paper explains why the same approach breaks down for polymorphic calculi and how to recover some of its simplicity.

Transfer hell is the colloquial term for being forced to apply type-changing lemmas to terms that are already known to be equal. The root problem is that substitutions that are propositionally equal are not definitionally equal, so terms must repeatedly be transported across equal types. This situation is annoying because proof assistants like Agda or Coq plainly refuse to let users even state this equality.¹ This phenomenon appears when substitutions themselves carry dependent typing information. In System F and F ω , expression substitutions depend on type substitutions and type substitutions depend on renamings; composing them yields types that are equal only propositionally, forcing transports throughout proofs. One workaround is to state definitions and proofs with explicit equivalence relations, but this practice is cumbersome, inefficient, and far away from the holy grail of definitional equality, which just works.

To our knowledge, previous intrinsically typed mechanization of System F ω (e.g., [?]) follow a natural design: expression syntax is indexed by typing derivations and substitutions are indexed by substitutions at the type level. While elegant at the level of definitions, this design makes proofs brittle. Because expression substitutions depend on type substitutions, and type substitutions depend on renamings, which again depend on type renamings, routine equalities require repeated transport across equal types, leading to directly to transfer hell.

¹A situation that can be addressed using heterogeneous equality introduced in Conor McBride's thesis [?]. [?] discuss this approach in the context of an adequacy proof for a logical relation.

The message of this pearl is simple: when intrinsically typed syntax becomes complicated, the problem is often not the encoding of terms but the definition of equality on substitutions. Strengthening definitional equality can simplify metatheory more effectively than adding automation or additional lemmas.

In this paper, we examine the transfer hell arising by intrinsically-typed syntax encodings of System F and System $F\omega$. Specifically, we consider the problems arising with type-renaming-indexed renamings and type-substitution-indexed substitutions. Our solution is to strengthen definitional equality rather than to add more lemmas. We make substitution compute canonically by internalizing the equational theory of substitutions (the σ -calculus) as definitional equality using Agda's rewriting mechanism. As a consequence, extensionally equal substitutions normalize to the same form and transports disappear from routine proofs. This approach is inspired by the tactics of Autosubst [??], but internalized by rewriting [?]. We build on a standard functional representation of renamings and substitutions [?], prove the lemmas underlying the equational laws of the sigma calculus, and then inject these equations into Agda's rewrite engine, i.e., make the sigma calculus equations definitional (for type substitutions). The technical challenge is to ensure that the added equations remain terminating and confluent together with the proof assistant's built-in definitional equality.

We first exercise this program for plain System F in Section 2, significantly simplifying the transfer-ridden approach used by [?], and then apply the same techniques to System $F\omega$ in Section 3. In the latter, the main difference to previous work [?] lies in our intrinsic treatment of type conversion, which leads to significant simplifications. In particular, in our developments, transport lemmas disappear, subject reduction holds by construction, and type conversion in System $F\omega$ is handled intrinsically.

In summary, intrinsically typed syntax fails for polymorphic calculi because substitutions lack canonical equality; making them compute canonically restores the original promise of intrinsic typing.

Contributions

This functional pearl demonstrates how to mechanize polymorphic calculi using intrinsically typed syntax without pervasive transport reasoning.

- We identify the fundamental cause of “transfer hell” in intrinsically typed encodings of System F and System $F\omega$: renamings and substitutions indexed by other substitutions produce equal types that are not definitionally equal.
- We show how to eliminate these transports by installing the equational laws of the σ -calculus for substitutions as definitional equality using Agda's rewriting mechanism, taking care to make the equations compatible with the built-in definitional equality.
- We demonstrate that the resulting rewrite system is compatible with Agda's definitional equality (in particular, terminating and confluent) for the functional representation of substitutions and verify that the resulting rewrite system is terminating and confluent with respect to the proof assistant's definitional equality.
- We mechanize parts of the metatheory of System F and extend it to System $F\omega$, obtaining substantially simpler proofs and an intrinsic treatment of type conversion for the latter.
- From this experience we extract practical guidelines for designing intrinsically typed syntax in dependently typed proof assistants.

```

148 - renamings
149  $\_ \rightarrow^R \_ : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Set}$ 
150  $n_1 \rightarrow^R n_2 = \text{Fin } n_1 \rightarrow \text{Fin } n_2$ 
151
152 opaque
153 - weakening
154  $\text{wk}^R : n \rightarrow^R \text{succ } n$ 
155  $\text{wk}^R = \text{succ}$ 
156
157 - identity renaming
158  $\text{id}^R : n \rightarrow^R n$ 
159  $\text{id}^R \alpha = \alpha$ 
160
161 - extend with new variable
162  $\_ \cdot^R \_ : \text{Fin } n_2 \rightarrow n_1 \rightarrow^R n_2 \rightarrow \text{succ } n_1 \rightarrow^R n_2$ 
163  $(\alpha \cdot^R \zeta) \text{zero} = \alpha$ 
164  $(\_ \cdot^R \zeta) (\text{succ } \alpha) = \zeta \alpha$ 
165
166 - apply renaming to variable
167  $\_ \&^R \_ : \text{Fin } n_1 \rightarrow n_1 \rightarrow^R n_2 \rightarrow \text{Fin } n_2$ 
168  $\alpha \&^R \zeta = \zeta \alpha$ 
169
170 - composition
171  $\_ \circ^R \_ : n_1 \rightarrow^R n_2 \rightarrow n_2 \rightarrow^R n_3 \rightarrow n_1 \rightarrow^R n_3$ 
172  $(\zeta_1 \circ^R \zeta_2) \alpha = \zeta_2 (\zeta_1 \alpha)$ 
173
174 - lifting
175  $\_ \uparrow^R : n_1 \rightarrow^R n_2 \rightarrow \text{succ } n_1 \rightarrow^R \text{succ } n_2$ 
176  $\_ \uparrow^R \zeta = \text{zero} \cdot^R (\zeta \circ^R \text{wk}^R)$ 
177
178 - apply renaming to type
179  $\_ [\ ]^R : \text{Type } n_1 \rightarrow n_1 \rightarrow^R n_2 \rightarrow \text{Type } n_2$ 
180  $(\alpha) [\ \zeta ]^R = (\alpha \&^R \zeta)$ 
181  $(\forall \alpha \ T) [\ \zeta ]^R = \forall \alpha \ (T [\ \zeta \uparrow^R ]^R)$ 
182  $(T_1 \Rightarrow T_2) [\ \zeta ]^R = (T_1 [\ \zeta ]^R) \Rightarrow (T_2 [\ \zeta ]^R)$ 
183
184
185
186
187
188
189
190
191
192
193
194
195
196

```

```

- substitutions
 $\_ \rightarrow^S \_ : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Set}$ 
 $n_1 \rightarrow^S n_2 = \text{Fin } n_1 \rightarrow \text{Type } n_2$ 

opaque
- lift renaming to substitution
 $\langle \_ \rangle : n_1 \rightarrow^R n_2 \rightarrow n_1 \rightarrow^S n_2$ 
 $\langle \zeta \rangle \alpha = (\alpha \&^R \zeta)$ 

- extend with new type
 $\_ \cdot^S \_ : \text{Type } n_2 \rightarrow n_1 \rightarrow^S n_2 \rightarrow \text{succ } n_1 \rightarrow^S n_2$ 
 $(T \cdot^S \eta) \text{zero} = T$ 
 $(T \cdot^S \eta) (\text{succ } \alpha) = \eta \alpha$ 

- apply substitution to variable
 $\_ \&^S \_ : \text{Fin } n_1 \rightarrow n_1 \rightarrow^S n_2 \rightarrow \text{Type } n_2$ 
 $\alpha \&^S \eta = \eta \alpha$ 

- lifting
 $\_ \uparrow^S : n_1 \rightarrow^S n_2 \rightarrow \text{succ } n_1 \rightarrow^S \text{succ } n_2$ 
 $\_ \uparrow^S \eta = (\text{zero} \cdot^S \lambda \alpha \rightarrow (\eta \alpha) [\ \text{wk}^R ]^R)$ 

- apply substitution to type
 $\_ [\ ]^S : \text{Type } n_1 \rightarrow n_1 \rightarrow^S n_2 \rightarrow \text{Type } n_2$ 
 $(\alpha) [\ \eta ]^S = \alpha \&^S \eta$ 
 $(\forall \alpha \ T) [\ \eta ]^S = \forall \alpha \ (T [\ \eta \uparrow^S ]^S)$ 
 $(T_1 \Rightarrow T_2) [\ \eta ]^S = (T_1 [\ \eta ]^S) \Rightarrow (T_2 [\ \eta ]^S)$ 

opaque
- composition
 $\_ \circ^S \_ : n_1 \rightarrow^S n_2 \rightarrow n_2 \rightarrow^S n_3 \rightarrow n_1 \rightarrow^S n_3$ 
 $(\eta_1 \circ^S \eta_2) \alpha = (\eta_1 \alpha) [\ \eta_2 ]^S$ 

```

Fig. 2. Renamings and substitutions on types

2 System F

2.1 Types

The syntax of System F types uses the standard de Bruijn encoding of variables by natural numbers. The index n stands for the number of type variables in scope. Variables are drawn from the type $\text{Fin } n$, i.e., the set $\{0, \dots, n-1\}$. The $\forall \alpha$ -binder introduces a new type variable, and $\Rightarrow$ is the infix function type constructor.

```

data Type (n : Nat) : Set where
  ' _ : Fin n → Type n

```

```

197   $\forall \alpha : \text{Type } (\text{succ } n) \rightarrow \text{Type } n$ 
198   $\_ \Rightarrow \_ : \text{Type } n \rightarrow \text{Type } n \rightarrow \text{Type } n$ 

```

Figure 2 contains the first step in defining type renamings (left column) and substitutions (right column). A type renaming ζ maps variables from one scope n_1 to another n_2 . We write the type of type renamings as $n_1 \rightarrow^R n_2$. The definitions comprise (in order) weakening, the identity renaming, extending a renaming, applying a renaming to a variable, composing two renamings, lifting a renaming (to move under a binder), and applying a renaming to a type.

At this point, the astute reader may wonder about the use of `opaque` in the definitions. The function symbols defined in an `opaque` block are usable from outside, but their computational behavior is hidden. Thus, the only normal definition is $\uparrow^R$ for lifting a renaming. This selective hiding is essential for our approach because we want to establish new definitional equalities (i.e., the laws of the sigma calculus with first-class renamings) on top of the thus defined symbols. These new equalities interfere in a bad way with the standard definitional equalities, thus we hide the latter. To prove these equalities, we open the opaque definitions locally and proceed as usual.

For substitutions (right column of Figure 2) we proceed analogously. A type substitution η maps variables from scope n_1 to types defined over scope n_2 . We write the type of type substitutions as $n_1 \rightarrow^S n_2$. The definitions (in order) comprise lifting a renaming to a substitution, extending a substitution, applying a substitution to variable, lifting a substitution (to move under a binder), applying a substitution to a type, and composing two substitutions.

Figure 3 gives the equational laws of the sigma calculus with first-class renamings [?]. All of these equalities are supported by proofs using the definitions from Figure 2, but the proofs are not included because they are standard.

The first group gives computation rules for renamings, in particular, how to apply the different forms of renamings to a variable. The second group presents interaction laws for renamings. Most of them deal with composition.

The third group presents computation rules for substitutions starting with the application of a substitution to a variable. There is one equation for each possible form of a substitution: extended substitution, lifted renaming, and composition of substitutions. The final equation in this group characterizes lifting in terms of extension, weakening, and composition.

The fourth group contains laws about composition of substitutions, which are mostly analogous to the corresponding laws for renamings.

The fifth group presents laws that govern the composition of different kinds of term traversal. All four combinations of applying renamings and substitutions to a type are covered.

The final group contains covers laws that mediate between substitutions and renamings. The extraction of renamings from substitutions in general requires a dedicated solving strategy [?] and we have omitted some additionally required coincidence laws from the figure.

Orienting these equations from left to right results in a confluent and terminating rewrite system that we install in Agda using its rewrite pragma [?]. From now on, all equations in Figure 3 are definitional!

As a sanity check, we show that lifting a renaming as well as the action of a renaming satisfy the functor laws. All of these lemmas hold definitionally, some correspond directly equations from the sigma calculus in Figure 3. The latter are marked with $*$.²

```

240 lift*-id : (idR {n}  $\uparrow^R$ )  $\equiv$  idR

```

```

241 lift*-id = refl

```

```

242 lift*-comp : ( $\zeta'$   $\circ^R$   $\zeta$ )  $\uparrow^R \equiv (\zeta' \uparrow^R) \circ^R (\zeta \uparrow^R)$ 

```

²The naming of the lemmas is taken from [?] to enable direct comparisons.

```

246 - computing renamings
247 'beta-ext-zero : zero &^R (α .^R ζ) ≡ α
248 'beta-ext-suc : suc α &^R (α' .^R ζ) ≡ α &^R ζ
249 'beta-id : α &^R id^R ≡ α
250 'beta-wk : α &^R wk^R ≡ suc α
251 'beta-comp : α &^R (ζ1 ∘^R ζ2) ≡ (α &^R ζ1) &^R ζ2
252 - interaction between renamings
253 'associativity : (ζ1 ∘^R ζ2) ∘^R ζ3 ≡ ζ1 ∘^R (ζ2 ∘^R ζ3)
254 'distributivity : (α .^R ζ1) ∘^R ζ2 ≡ (α &^R ζ2) .^R (ζ1 ∘^R ζ2)
255 'interact : wk^R ∘^R (α .^R ζ) ≡ ζ
256 'comp-idr : ζ ∘^R id^R ≡ ζ
257 'comp-idl : id^R ∘^R ζ ≡ ζ
258 'η-id : zero {n} .^R wk^R ≡ id^R
259 'η-law : (zero &^R ζ) .^R (wk^R ∘^R ζ) ≡ ζ
260
261 - computing substitutions
262 beta-ext-zero : zero &^S (T .^S η) ≡ T
263 beta-ext-suc : suc α &^S (T .^S η) ≡ α &^S η
264 beta-rename : α &^S ⟨ ζ ⟩ ≡ (α &^R ζ)
265 beta-comp : α &^S (η1 ∘^S η2) ≡ (α &^S η1) [ η2 ]^S
266 beta-lift : η ↑^S ≡ ('zero) .^S (η ∘^S wk^R)
267 - interaction between substitutions
268 associativity : (η1 ∘^S η2) ∘^S η3 ≡ η1 ∘^S (η2 ∘^S η3)
269 distributivity : (T .^S η1) ∘^S η2 ≡ (T [ η2 ]^S) .^S (η1 ∘^S η2)
270 interact : ⟨ wk^R ⟩ ∘^S (T .^S η) ≡ η
271 comp-idr : η ∘^S ⟨ id^R ⟩ ≡ η
272 comp-idl : ⟨ id^R ⟩ ∘^S η ≡ η
273 η-id : ('zero {n}) .^S ⟨ wk^R ⟩ ≡ ⟨ id^R ⟩
274 η-law : (zero &^S η) .^S (⟨ wk^R ⟩ ∘^S η) ≡ η
275
276 - composing renamings and substitutions
277 identityr : T [ id^R ]^R ≡ T
278 compositionalityRS : (T [ ζ1 ]^R) [ η2 ]^S ≡ T [ ⟨ ζ1 ⟩ ∘^S η2 ]^S
279 compositionalityRR : (T [ ζ1 ]^R) [ ζ2 ]^R ≡ T [ ζ1 ∘^R ζ2 ]^R
280 compositionalitySR : (T [ η1 ]^S) [ ζ2 ]^R ≡ T [ η1 ∘^S ⟨ ζ2 ⟩ ]^S
281 compositionalitySS : (T [ η1 ]^S) [ η2 ]^S ≡ T [ η1 ∘^S η2 ]^S
282
283 - transforming substitutions to renamings
284 coincidence : T [ ⟨ ζ ⟩ ]^S ≡ T [ ζ ]^R
285 coincidence-comp : ⟨ ζ1 ⟩ ∘^S ⟨ ζ2 ⟩ ≡ ⟨ ζ1 ∘^R ζ2 ⟩
286

```

Fig. 3. Equational laws of the sigma calculus with first class renamings

```

295 weaken : Type n → Type (suc n)
296 weaken T = T [ wkR ]R
297
298 _[_]* : Type (suc n) → Type n → Type n
299 T [ T' ]* = T [ T' .S idS ]S
300
301 data Ctx : Nat → Set where
302   () : Ctx zero
303   _▷_ : Ctx n → Type n → Ctx n
304   _▷* : Ctx n → Ctx (suc n)
305
306 data _≡_ : Ctx n → Type n → Set where
307   zero : (Γ ▷ T) ≡ T
308   suc : Γ ≡ T → (Γ ▷ T') ≡ T'
309   suc* : Γ ≡ T → (Γ ▷*) ≡ (weaken T)
310
311
312 data Expr : Ctx n → Type n → Set where
313   ' _ : Γ ≡ T →
314     Expr Γ T
315   λx : Expr (Γ ▷ T1) T2 →
316     Expr Γ (T1 ⇒ T2)
317   _·_ : Expr Γ (T1 ⇒ T2) →
318     Expr Γ T1 →
319     Expr Γ T2
320   Λα : Expr (Γ ▷*) T →
321     Expr Γ (∀α T)
322   _·*_ : Expr Γ (∀α T) →
323     (T' : Type n) →
324     Expr Γ (T [ T' ]*)

```

Fig. 4. Type contexts, variables, and expressions

```

314 lift*-comp = refl
315
316 ren*-id : T [ idR ]R ≡ T
317 ren*-id = refl - *
318
319 ren*-comp : T [ ζ' ;R ζ ]R ≡ (T [ ζ' ]R) [ ζ ]R
320 ren*-comp = refl - *

```

We also show analogous results for substitution.

```

322 lifts*-id : (idS {n} ↑S) ≡ idS
323 lifts*-id = refl
324
325 lifts*-comp : (η' ;S η) ↑S ≡ (η' ↑S) ;S (η ↑S)
326 lifts*-comp = refl
327
328 sub*-id : T [ idS ]S ≡ T
329 sub*-id = refl
330
331 sub*-var : (' α) [ η ]S ≡ α &S η
332 sub*-var = refl - *
333
334 sub*-comp : T [ η ;S η' ]S ≡ (T [ η ]S) [ η' ]S
335 sub*-comp = refl - *

```

2.2 Expressions

Working towards intrinsically-typed expressions, Figure 4 defines type weakening and single substitution along with type contexts, expression variables, and the syntax of expressions. Type contexts and expression variables follow the work of [?]. Type contexts `Ctx` keep track of expression variables and type variables. Correspondingly, variables are implemented as typed de Bruijn indices with an extra case `suc*` to skip over type variables in the context. Skipping requires weakening of the type as it is used under one more type binding.

Expressions are indexed by a context and a type. Each expression constructor corresponds to a typing rule in System F. The rule for type application relies on single substitution for types.

The definition of a small-step operational semantics requires expression renamings and substitution to implement beta reduction for value abstractions and type abstractions. As before, we aim to define a call-by-name semantics for concision.

2.3 Expression Renaming and Substitution

For expressions, we represent renamings and substitutions directly by suitable functions, rather than axiomatic as with type renamings and substitutions. Hence, we cannot automate the simplification of expression substitutions; nevertheless, we define the primitives analogously.

2.3.1 Renaming. Expression renamings are indexed by type renamings because type variables in expression may have to be renamed at the same time as expression variables. This complication makes the types of the operations quite involved, although the actual operations are not that complicated.

$$\begin{aligned} _ \mid \Rightarrow^R _ : n_1 \rightarrow^R n_2 \rightarrow \text{Ctx } n_1 \rightarrow \text{Ctx } n_2 \rightarrow \text{Set} \\ \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 = \forall T \rightarrow \Gamma_1 \ni T \rightarrow \Gamma_2 \ni (T [\zeta]^R) \end{aligned}$$

Defining the identity renaming is straightforward due to the presence of rewriting for type substitutions.

$$\begin{aligned} \text{id}^R : \text{id}^R \mid \Gamma \Rightarrow^R \Gamma \\ \text{id}^R _ x = x \end{aligned}$$

Weakening of a renaming by a value binding.

$$\begin{aligned} \text{Wk}^R : \forall T \rightarrow \text{id}^R \mid \Gamma \Rightarrow^R (\Gamma \triangleright T) \\ \text{Wk}^R _ _ = \text{succ} \end{aligned}$$

Weakening of a renaming by a type binding.

$$\begin{aligned} \text{wk}^{R*} : \text{wk}^R \mid \Gamma \Rightarrow^R (\Gamma \triangleright^*) \\ \text{wk}^{R*} _ x = \text{succ}^* x \end{aligned}$$

Extension of a renaming.

$$\begin{aligned} _ \mid \cdot^R _ : \forall \zeta \rightarrow \Gamma_2 \ni (T [\zeta]^R) \rightarrow \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 \rightarrow \zeta \mid (\Gamma_1 \triangleright T) \Rightarrow^R \Gamma_2 \\ (_ \mid x \cdot^R \rho) _ \text{zero} = x \\ (_ \mid \cdot^R \rho) _ (\text{succ } x) = \rho _ x \end{aligned}$$

Lifting of a renaming over a value binding.

$$\begin{aligned} _ \mid \uparrow^R _ : \forall \zeta \rightarrow \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 \rightarrow \forall T \rightarrow \zeta \mid (\Gamma_1 \triangleright T) \Rightarrow^R (\Gamma_2 \triangleright (T [\zeta]^R)) \\ (\zeta \mid \rho \uparrow^R _) = \zeta \mid \text{zero} \cdot^R (\zeta, \text{id}^R \mid \rho \uparrow^R (\text{Wk}^R _)) \end{aligned}$$

Lifting of a renaming over a type binding.

$$\begin{aligned} _ \mid \uparrow^{R*} : \forall \zeta \rightarrow \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 \rightarrow (\zeta \uparrow^R) \mid (\Gamma_1 \triangleright^*) \Rightarrow^R (\Gamma_2 \triangleright^*) \\ (_ \mid \rho \uparrow^{R*}) _ (\text{succ}^* x) = \text{succ}^* (\rho _ x) \end{aligned}$$

Inductive definition of a renaming on expressions. The last case of the definition requires rewriting to type check.

opaque

$$\begin{aligned} _ \mid _ \mid^R : (\zeta : n_1 \rightarrow^R n_2) \rightarrow \text{Expr } \Gamma_1 \ T \rightarrow \zeta \mid \Gamma_1 \Rightarrow^R \Gamma_2 \rightarrow \text{Expr } \Gamma_2 \ (T [\zeta]^R) \\ \zeta \mid (\text{' } x) [\rho]^R = \text{' } (\zeta \mid x \cdot^R \rho) \end{aligned}$$

$$\begin{aligned}
 & _ | (\lambda x \ e) [\rho]^R = \lambda x \ (_ | e [\rho \uparrow^R _]^R) \\
 & _ | (\Lambda \alpha \ e) [\rho]^R = \Lambda \alpha \ (_ | e [\uparrow^{R*} \rho]^R) \\
 & _ | (e_1 \cdot e_2) [\rho]^R = (_ | e_1 [\rho]^R) \cdot (_ | e_2 [\rho]^R) \\
 & \zeta | (e \cdot^* T) [\rho]^R = (\zeta | e [\rho]^R) \cdot^* (T [\zeta]^R)
 \end{aligned}$$

2.3.2 *Substitution*. Expression substitutions are indexed by type substitutions.

$$\begin{aligned}
 & _ | \Rightarrow^S _ : n_1 \rightarrow^S n_2 \rightarrow \text{Ctx } n_1 \rightarrow \text{Ctx } n_2 \rightarrow \text{Set} \\
 & \eta | \Gamma_1 \Rightarrow^S \Gamma_2 = \forall T \rightarrow \Gamma_1 \ni T \rightarrow \text{Expr } \Gamma_2 \ (T [\eta]^S)
 \end{aligned}$$

The identity substitution for expressions is straightforward to define. But this simplicity is due to the installed rewriting of type substitutions, which avoids an explicit application of a transport lemma using `comp-idr`.

$$\begin{aligned}
 & \text{id}^S : \text{id}^S | \Gamma \Rightarrow^S \Gamma \\
 & \text{id}^S _ = _
 \end{aligned}$$

Extending a substitution is standard.

$$\begin{aligned}
 & _ | \cdot^S _ : \forall \eta \rightarrow \text{Expr } \Gamma_2 \ (T [\eta]^S) \rightarrow \eta | \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow \eta | (\Gamma_1 \triangleright T) \Rightarrow^S \Gamma_2 \\
 & (_ | e \cdot^S \sigma) _ \text{zero} = e \\
 & (_ | e \cdot^S \sigma) _ (\text{suc } x) = \sigma _ x
 \end{aligned}$$

Due to the structure of contexts, we need two lifting lemmas, one for value bindings and another for type bindings. The former requires a transport lemma, which is applied by rewriting.

$$\begin{aligned}
 & _ | \uparrow^S _ : \forall \eta \rightarrow \eta | \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow \forall T \rightarrow \eta | (\Gamma_1 \triangleright T) \Rightarrow^S (\Gamma_2 \triangleright (T [\eta]^S)) \\
 & \eta | \sigma \uparrow^S T = \eta | (_ \text{zero}) \cdot^S \lambda _ x \rightarrow \text{id}^R | (\sigma _ x) [\text{Wk}^R _]^R
 \end{aligned}$$

Lifting a substitution over a type binding is straightforward.

$$\begin{aligned}
 & _ | \uparrow^{S*} _ : \forall \eta \rightarrow \eta | \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow (\eta \uparrow^S) | (\Gamma_1 \triangleright^*) \Rightarrow^S (\Gamma_2 \triangleright^*) \\
 & (\eta | \sigma \uparrow^{S*}) _ (\text{suc}^* x) = _ | (\sigma _ x) [\text{wk}^{R*}]^R
 \end{aligned}$$

These operations are sufficient to define the application of a substitution to an expression.

opaque

$$\begin{aligned}
 & _ | _ |^S : (\eta : n_1 \rightarrow^S n_2) \rightarrow \text{Expr } \Gamma_1 \ T \rightarrow \eta | \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow \text{Expr } \Gamma_2 \ (T [\eta]^S) \\
 & \eta | (_ x) [\sigma]^S = \eta | x \&^S \sigma \\
 & \eta | (\lambda x \ e) [\sigma]^S = \lambda x \ (\eta | e [\eta | \sigma \uparrow^S _]^S) \\
 & \eta | (\Lambda \alpha \ e) [\sigma]^S = \Lambda \alpha \ ((\eta \uparrow^S) | e [\eta | \sigma \uparrow^{S*}]^S) \\
 & \eta | (e \cdot e_1) [\sigma]^S = (\eta | e [\sigma]^S) \cdot (\eta | e_1 [\sigma]^S) \\
 & \eta | (e \cdot^* T) [\sigma]^S = (\eta | e [\sigma]^S) \cdot^* (T [\eta]^S)
 \end{aligned}$$

2.4 Semantics

We are now in position to define a small-step operational semantics. The definition of single substitution, needed for beta reduction, is analogous to the one on types: we build the substitution by extending the identity substitution with the argument. In addition, a type substitution into expression is needed for beta reduction of type applications. This substitution is also implemented by an expression substitution indexed by a type substitution that performs the actual change.

$$\begin{aligned}
 & _ | : \text{Expr } (\Gamma \triangleright T') \ T \rightarrow \text{Expr } \Gamma \ T' \rightarrow \text{Expr } \Gamma \ T \\
 & e [_]' = \text{id}^S | e [\text{id}^S | e' \cdot^S \text{id}^S]^S
 \end{aligned}$$

```

442 data Value : Expr  $\Gamma$   $T \rightarrow$  Set where
443    $\lambda x : (e : \text{Expr } (\Gamma \triangleright T_1) T_2) \rightarrow \text{Value } (\lambda x e)$ 
444    $\Lambda \alpha : \text{Value } e \rightarrow \text{Value } (\Lambda \alpha e)$ 
445
446 data Progress : Expr  $\Gamma$   $T \rightarrow$  Set where
447   done : ( $v : \text{Value } e$ )  $\rightarrow$  Progress  $e$ 
448   step : ( $e \rightarrow e' : e \rightarrow e'$ )  $\rightarrow$  Progress  $e$ 
449
450 NoVar : Ctx  $n \rightarrow$  Set
451 NoVar  $\emptyset = \top$ 
452 NoVar ( $\Gamma \triangleright T$ ) =  $\perp$ 
453 NoVar ( $\Gamma \triangleright^*$ ) = NoVar  $\Gamma$ 
454
455 noVar : NoVar  $\Gamma \rightarrow \neg (\Gamma \ni T)$ 
456 noVar  $nv (\text{suc}^* x) = \text{noVar } nv x$ 
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490

```

$\text{progress} : \text{NoVar } \Gamma \rightarrow (e : \text{Expr } \Gamma T) \rightarrow \text{Progress } e$
 $\text{progress } nv ('x) = \perp\text{-elim } (\text{noVar } nv x)$
 $\text{progress } nv (\lambda x e) = \text{done } (\lambda x e)$
 $\text{progress } nv (e \cdot e_1)$
 with progress $nv e$
 ... | done $(\lambda x e_2) = \text{step } \beta\text{-}\lambda$
 ... | step $e \rightarrow e' = \text{step } (\xi\text{-} e \rightarrow e')$
 $\text{progress } nv (\Lambda \alpha e)$
 with progress $nv e$
 ... | done $v = \text{done } (\Lambda \alpha v)$
 ... | step $e \rightarrow e' = \text{step } (\xi\text{-}\Lambda e \rightarrow e')$
 $\text{progress } nv (e \cdot^* T)$
 with progress $nv e$
 ... | done $(\Lambda \alpha v) = \text{step } \beta\text{-}\Lambda$
 ... | step $e \rightarrow e' = \text{step } (\xi\text{-}^* e \rightarrow e')$

Fig. 5. Progress for System F

```

462  $\_ [ * \_ ] : \text{Expr } (\Gamma \triangleright^* ) T \rightarrow (T' : \text{Type } n) \rightarrow \text{Expr } \Gamma (T [ T' ]^*)$ 
463  $e [ * T' ] = (T' \cdot^S \text{id}^S) \mid e [ \text{id}^S \mid T' \cdot^{S*} \text{id}^S ]^S$ 
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490

```

With substitution in place, the definition of call-by-name small-step reduction is straightforward. As usual, such a definition of reduction for intrinsically-typed syntax implies subject reduction by construction.

```

468 data  $\_ \rightarrow \_ : \text{Expr } \Gamma T \rightarrow \text{Expr } \Gamma T \rightarrow$  Set where
469    $\beta\text{-}\lambda : (\lambda x e_1 \cdot e_2) \rightarrow (e_1 [ e_2 ])$ 
470    $\beta\text{-}\Lambda : (\Lambda \alpha e \cdot^* T) \rightarrow (e [ * T' ]^*)$ 
471    $\xi\text{-}\cdot : e_1 \rightarrow e_1' \rightarrow (e_1 \cdot e_2) \rightarrow (e_1' \cdot e_2)$ 
472    $\xi\text{-}\cdot^* : e \rightarrow e' \rightarrow (e \cdot^* T) \rightarrow (e' \cdot^* T)$ 
473    $\xi\text{-}\Lambda : e \rightarrow e' \rightarrow (\Lambda \alpha e) \rightarrow (\Lambda \alpha e')$ 
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490

```

To obtain type safety, it remains to prove progress. Figure 5 presents the complete proof of progress for call-by-name System F. A value is either a lambda or a big lambda where the body is already a value. An expression fulfills progress if it is either a value (**done**) or it can make a step (**step**). Due to the definition of value for big lambdas, we establish progress for type contexts that may contain type bindings, but no value bindings. These contexts are characterized by function **NoVar**. The actual proof of progress is by induction on the structure of expressions. The overall structure of this proof closely follows the corresponding proof by [?]. Their proof is more complicated because their semantics is call-by-value and their language supports isorecursive types.

2.5 Laws for Renamings and Substitutions

Proving type safety for System F *requires* exercising the substitution theory at the *type level*. Without installing its laws as definitional equalities, many definitions become more cumbersome because transport lemmas have to be inserted to equalize the types of (Agda) expressions.

While proving type safety does *not* require exercising same theory at the *expression level*, it is still good practice to verify functorial and monadic properties of expression substitutions. Moreover,

if we were to move on to constructing and proving properties of logical relations, then these properties would be needed [?].

As an example, we examine the composition of two expression substitutions. Its definition involves two type substitutions η_1 and η_2 that index the corresponding expression substitutions σ_1 and σ_2 . The composed expression substitution is indexed by the composition of η_1 and η_2 . Type checking this definition already exercises the laws for composition of type substitutions.

opaque

$$\begin{aligned} _ , _ \circ^S _ : \forall \eta_1 \eta_2 \rightarrow \eta_1 \mid \Gamma_1 \Rightarrow^S \Gamma_2 \rightarrow \eta_2 \mid \Gamma_2 \Rightarrow^S \Gamma_3 \rightarrow (\eta_1 \circ^S \eta_2) \mid \Gamma_1 \Rightarrow^S \Gamma_3 \\ (_ , _ \mid \sigma_1 \circ^S \sigma_2) _ x = _ \mid (\sigma_1 _ x) [\sigma_2]^S \end{aligned}$$

Here is the statement of functoriality of expression substitution: applying two substitutions one after the other to an expression is the same as applying their composition. This statement is a very prominent example of transfer hell: the types of the left and right side of the equality are different and would require a transfer without our rewriting machinery in place.

$$\begin{aligned} \text{Compositionality}^{SS} : \forall (e : \text{Expr } \Gamma_1 \ T) (\eta_1 : n_1 \rightarrow^S n_2) (\eta_2 : n_2 \rightarrow^S n_3) (\sigma_1 : \eta_1 \mid \Gamma_1 \Rightarrow^S \Gamma_2) (\sigma_2 : \eta_2 \mid \Gamma_2 \Rightarrow^S \Gamma_3) \\ \eta_2 \mid (\eta_1 \mid e [\sigma_1]^S) [\sigma_2]^S \equiv (\eta_1 \circ^S \eta_2) \mid e [\eta_1 , \eta_2 \mid \sigma_1 \circ^S \sigma_2]^S \end{aligned}$$

We show the proof of this property just do demonstrate that type adjustments for type substitutions can be ignored and we can concentrate on the gist of the proof, rather than being overwhelmed by transfers. The remaining fly in the ointment is the explicit passing of the type substitutions η_1 and η_2 . We chose to make them explicit; while Agda can infer them in many places, there are equally many places where inference is not possible.

$$\text{Compositionality}^{SS} (_ x) \quad \eta_1 \eta_2 \sigma_1 \sigma_2 = \text{refl}$$

$$\begin{aligned} \text{Compositionality}^{SS} (\lambda x \ e) \quad \eta_1 \eta_2 \sigma_1 \sigma_2 &= \text{cong } \lambda x \ (\text{begin} \\ &_ \equiv \langle \text{Compositionality}^{SS} \ e \ \eta_1 \ \eta_2 \ (\eta_1 \mid \sigma_1 \uparrow^S _) (\eta_2 \mid \sigma_2 \uparrow^S _) \rangle \\ &_ \equiv \langle \text{cong } ((\eta_1 \circ^S \eta_2) \mid e [_]^S) (\text{Lift-Dist-Comp}^{SS} \ \sigma_1 \ \sigma_2) \rangle \\ &_ \square) \end{aligned}$$

$$\begin{aligned} \text{Compositionality}^{SS} (\lambda \alpha \ e) \quad \eta_1 \eta_2 \sigma_1 \sigma_2 &= \text{cong } \lambda \alpha \ (\text{begin} \\ &_ \equiv \langle \text{Compositionality}^{SS} \ e \ (\eta_1 \uparrow^S) (\eta_2 \uparrow^S) (\eta_1 \mid \sigma_1 \uparrow^{S*}) (\eta_2 \mid \sigma_2 \uparrow^{S*}) \rangle \\ &_ \equiv \langle \text{cong } (((\eta_1 \circ^S \eta_2) \uparrow^S) \mid e [_]^S) (\text{lift}^* \text{-dist-Comp}^{SS} \ \eta_1 \ \eta_2 \ \sigma_1 \ \sigma_2) \rangle \\ &_ \square) \end{aligned}$$

$$\text{Compositionality}^{SS} (e_1 \cdot e_2) \ \eta_1 \ \eta_2 \ \sigma_1 \ \sigma_2 = \text{cong}_2 \ _ _ (\text{Compositionality}^{SS} \ e_1 \ \eta_1 \ \eta_2 \ \sigma_1 \ \sigma_2) (\text{Compositionality}^{SS} \ e_2 \ \eta_1 \ \eta_2 \ \sigma_1 \ \sigma_2)$$

$$\text{Compositionality}^{SS} (e \cdot^* T) \ \eta_1 \ \eta_2 \ \sigma_1 \ \sigma_2 = \text{cong} \ (_ \cdot^* (T [\eta_1 \circ^S \eta_2]^S)) (\text{Compositionality}^{SS} \ e \ \eta_1 \ \eta_2 \ \sigma_1 \ \sigma_2)$$

This proof relies on further properties, in particular **Lift-Dist-Comp**^{SS} (lifting a composition of substitutions is equal to composing their liftings) and **lift*-dist-Comp**^{SS} (its counterpart for lifting over type bindings), where the statement and proof would require explicit transfers. Essentially, these properties form a “food chain” that stops at the composition properties of renamings with renamings and liftings, all with similar issues. The supplemental material gives the full story.

2.6 Comparison to Proof in Transfer Hell

We compare this proof to the one without the additional definitional equalities for type substitutions, provided in the supplement for the aforementioned logical relation construction [?]. Here we only state the *type* of the proof and the induction case for application, which is among the short ones. The authors use heterogeneous equality to proof the lemma, thereby avoiding some of the troubles arising from transfer hell. The full proof can be found in Appendix A

	Rewriting		Transfer Hell	
	LOC	Chars	LOC	Chars
Type substitution + proofs	217	12 246	285	12 584
Expression substitution + proofs	199	11 465	1 405	69 846
Total	416	23 711	1 690	82 430
Type-check time (wall)	4.49 s		9.51 s	

Fig. 6. Code transitively required to prove the four expression-level compositionality lemmas in two Agda formalizations of System F. LOC counts non-blank, non-comment lines; type-check times are wall-clock, cold cache.

2.7 Examples

As an example of System F expressions, we present some definitions for Church numerals. We start with their type. For readability, we define aliases for type variables like α for Z , β for $(S Z)$, and analogously for expression variables and binders.

```

 $N^c : \text{Type } 0 *$ 
 $N^c = \forall \alpha ((\alpha \Rightarrow \alpha) \Rightarrow (\alpha \Rightarrow \alpha))$ 

```

We give the definitions for some constants and the successor operation.

```

 $\text{zero}^c : \text{Expr } 0 N^c$ 
 $\text{zero}^c = \lambda \alpha (\lambda g (\lambda x 'x))$ 

 $\text{one}^c : \text{Expr } 0 N^c$ 
 $\text{one}^c = \lambda \alpha (\lambda g (\lambda x ('g \cdot 'x)))$ 

 $\text{succ}^c : \text{Expr } 0 (N^c \Rightarrow N^c)$ 
 $\text{succ}^c = \lambda x (\lambda \alpha (\lambda g (\lambda x ('g \cdot ((((' (suc (suc (suc^* \text{zero})))) \cdot * \alpha) \cdot 'g) \cdot 'x))))))$ 

 $\text{two}^c : \text{Expr } 0 N^c$ 
 $\text{two}^c = \text{succ}^c \cdot (\text{succ}^c \cdot \text{zero}^c)$ 

```

3 System F_ω

System F_ω extends System F with higher-kinded types, that is, abstraction over type functions. This calculus is the basis of Haskell's type system where type functions are limited to partial application of type constructors. Abstraction of higher-kinded types is key to defining type classes for functors, monads, etc. An extension of F_ω with isorecursive types is the basis for Plutus, the language of the Cardano blockchain [?].

3.1 Definitions

We only give a glimpse of the definitions of the syntax of types and expressions, which are taken from [?] and hence generalize the definitions for System F from Section 2. We do not reiterate the development of renamings and substitutions, as it transfers mutatis mutandis: the typings get more involved, but the equational laws remain exactly the same.

For System F_ω , type variables have more structure as in System F, as there are arrow kinds in addition to plain types. We use I, J, K to range over types.

```
589 data Kind : Set where
```

```
590   * : Kind
```

```
591   _⇒_ : Kind → Kind → Kind
```

592 Consequently, kind contexts are not mere numbers, but lists of kinds that grow to the right. Kind
593 contexts are ranged over by Φ, Ψ, Θ .

```
595 data Ctx* : Set where
```

```
596   ∅ : Ctx*
```

```
597   _▷* _ : Ctx* → Kind → Ctx*
```

598 Type variables have the same structure as plain variables in the simply-typed lambda calculus (see
599 Figure 1).

```
601 data _∃* _ : Ctx* → Kind → Set where
```

```
602   zero : (Φ ▷* K) ∃* K
```

```
603   suc : Φ ∃* K → (Φ ▷* J) ∃* K
```

604 The definition of the type syntax also mimicks the expressions of the simply-typed lambda
605 calculus, but with an important twist, which deviates from the prior work.

606 As System $F\omega$ enjoys full beta reduction at the type level, we do not define application (with symbol
607 $\$$) as a data constructor. Instead, we introduce it as a symbol and use rewriting to extend functions
608 on the type syntax to include application.

609 As an example, we show the definition of applying a renaming to an expression. We elide the
610 preceding definitions of lifting, extension, and the type substitution machinery because they are
611 identical to the ones for System F in Section 2.1, save for the more elaborate type signatures.

```
615 _|_|R : (ζ : Φ →R Ψ) → Expr Γ1 T → ζ | Γ1 ⇒R Γ2 → Expr Γ2 (T [ ζ ]R)
```

```
616 _| (x) [ ρ ]R = (ρ _ x)
```

```
617 _| (λx e) [ ρ ]R = λx ( _ | e [ ρ ↑R _ ]R)
```

```
618 _| (λα e) [ ρ ]R = λα ( _ | e [ _ | ρ ↑R* _ ]R)
```

```
619 _| (e1 · e2) [ ρ ]R = ( _ | e1 [ ρ ]R) · ( _ | e2 [ ρ ]R)
```

```
620 ζ | (e.* T) [ ρ ]R = (ζ | e [ ρ ]R).*(T [ ζ ]R)
```

621 On top of the rewriting theory for type substitutions, we install type-level beta reduction as a
622 rewrite rule, using single type substitution on the right side:

```
624 postulate
```

```
625   β≡* : ∀ {Φ J K} {T1 : Type (Φ ▷* J) K} {T2 : Type Φ J} →
```

```
626     _$ _ {Φ}{J}{K} (λα {Φ}{J}{K} T1) T2 ≡ T1 [ T2 ]*
```

```
628 {-# REWRITE β≡* #-}
```

629 The combined theory is still confluent and terminating [? ?].

630 Surprisingly, type-level beta reduction is not required to define the syntax of expressions, ex-
631 pression renamings and substitutions, defining the operational semantics, as well as the proof of
632 type safety. Thanks to definitional beta conversion at the type level, the expression syntax does
633 *not* require an explicit type conversion rule as it was the case in prior work [?]. Hence, the proof
634 presented in Figure 5 for System F carries over, mutatis mutandis (i.e., adapting the type signatures).

635 In the next section, we develop some examples that genuinely rely on type conversion.

3.2 Examples

?] consider several examples to demonstrate their encoding of System $F\omega\mu$. Their first group of examples covers Church numerals in System F. These are covered in Section 2.7. Their second group of example covers recursive types. We have no counterpart for them. Unfortunately, they do not provide examples that exercise the full power of System $F\omega$, namely higher-kinded types! We fill this gap in the following.

3.2.1 Church Numerals. First, we encode (simply-typed) Church numerals *in the type language*. As beta reduction on the type level is definitional in our development, normalization of Church numerals is performed by Agda's type checker.

The kind of type level Church numerals is as expected.

$\mathbb{N}^k : \text{Kind}$

$\mathbb{N}^k = (* \Rightarrow *) \Rightarrow (* \Rightarrow *)$

We define zero and successor for Church numerals in the usual way. For readability, we define aliases like $\lambda\beta$ and so on for the type level lambda $\lambda\alpha$. Analogously, we define aliases for type variables like α for α , β for β , and so on.

$\text{zero}^k : \text{Type } \emptyset \mathbb{N}^k$

$\text{zero}^k = \lambda\beta (\lambda\alpha \alpha)$

$\text{succ}^k : \text{Type } \emptyset (\mathbb{N}^k \Rightarrow \mathbb{N}^k)$

$\text{succ}^k = \lambda\gamma (\lambda\beta (\lambda\alpha (\beta \$ (\gamma \$ \alpha))))$

We construct the Church numeral for 2 by applying the successor twice to zero. The proof that this term is equal to its normal form is definitional.

$\text{two}^k : \text{Type } \emptyset \mathbb{N}^k$

$\text{two}^k = \text{succ}^k \$ (\text{succ}^k \$ \text{zero}^k)$

$_ : \text{two}^k \equiv \lambda\beta (\lambda\alpha (\beta \$ (\beta \$ \alpha)))$

$_ = \text{refl}$

Finally, we define addition and construct 2 by adding 1 + 1. The corresponding equality proof puts a significant load on the type checker and takes more than a minute to complete on the authors' machine.

$\text{one}^k : \text{Type } \emptyset \mathbb{N}^k$

$\text{one}^k = \lambda\beta (\lambda\alpha (\beta \$ \alpha))$

$\text{add}^k : \text{Type } \emptyset (\mathbb{N}^k \Rightarrow (\mathbb{N}^k \Rightarrow \mathbb{N}^k))$

$\text{add}^k = \lambda\alpha (\lambda\beta (\lambda\gamma (\lambda\delta (((\text{succ} (\text{succ} (\text{succ} \text{zero}))) \$ (\text{succ} \text{zero})) \$ (((\text{succ} (\text{succ} \text{zero})) \$ (\text{succ} \text{zero})) \$ (\text{zero}))))))$

$_ : \text{two}^k \equiv (\text{add}^k \$ \text{one}^k) \$ \text{one}^k$

$_ = \text{refl}$

3.2.2 Arrows. The final example is inspired by the work on arrows [?]. This work uses type classes to abstract over different instantiations of the function arrow. For clarity, we start with an Agda version of the function that we wish to implement in System $F\omega$.

The function **abst** is a glorified identity function. It abstracts over a binary type constructor κ , two types β and α , then it takes an abstract application function *app* and applies it to an abstract function

f and a suitable argument x . Function `inst` instantiates this function. It acts like an eliminator for `abst`. The function `use` puts both together.

```

687 ty : Set1
688 ty = (κ : Set → Set → Set) (β α : Set) (app : κ β α → β → α) (f : κ β α) (x : β) → α
689
690 abst : ty
691 abst = λ κ β α → λ app f x → app f x
692
693 inst : ∀ α → ty → α → α
694 inst = λ α g x → g (λ β α → (β → α)) α α (λ f x → f x) (λ z → z) x
695
696 use : ∀ α → α → α
697 use = λ α x → inst α abst x
698
699 use = λ α x → inst α abst x

```

We start by defining the type in our encoding.

```

701 ty-enc : Type Φ *
702 ty-enc = ∀κ (∀β (∀α (((κ $ 'β) $ 'α) ⇒ ('β ⇒ 'α))
703           ⇒ (((κ $ 'β) $ 'α) ⇒ ('β ⇒ 'α)))))
704

```

Next, we encode the defining expression equivalent to `abst`.

```

705
706 abst-enc : Expr Γ ty-enc
707 abst-enc = Λκ (Λβ (Λα (λf (λx (('f · 'y) · 'x)))))
708

```

To use this function, we encode functions `inst` and `use`.

```

709
710 inst-enc : Expr Γ (∀α (ty-enc ⇒ ('α ⇒ 'α)))
711 inst-enc = Λα (λg (λx (((('g ·* (λβ (λα ('β ⇒ 'α)))) ·* 'α) ·* 'α)
712           · (λy (λx ('y · 'x))))
713           · λx 'x) - (λx 'x))
714           · 'x)))
715

```

```

716 use-enc : Expr ∅ (∀α ('α ⇒ 'α))
717 use-enc = Λα (λx (((inst-enc ·* 'α) · abst-enc) · 'x))
718

```

The definitions of the encodings require beta conversion at the type level. These definitions are not accepted without the rewriting of beta reduction $\beta \equiv^*$.

4 Conclusion

Intrinsically typed syntax promises elegant mechanizations by making well-typed programs representable only as well-typed data, but for polymorphic calculi this promise is often undermined by pervasive transport reasoning. Our experience with System F and System F ω shows that the difficulty does not stem from binding structure or indexing itself, but from a mismatch between substitution and definitional equality: substitutions that are extensionally equal fail to compute to a canonical form. By internalizing the equational laws of the σ -calculus as definitional equality, substitutions become canonical and routine transports disappear from proofs. The resulting developments recover the simplicity seen in the simply-typed case while scaling to higher-order polymorphism and intrinsic type conversion. More broadly, this pearl suggests a practical guideline for mechanizing type systems in dependently typed proof assistants: when intrinsically typed encodings become proof-heavy, a promising strategy is often to strengthen definitional equality rather than introduce additional lemmas or automation.

A Proof for Compose Monad Law in Transfer Hell

736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784